

КРАВЧЕНКО А.С.

**ПРОГРАМНАЯ ДОКУМЕНТАЦИЯ НА
“ОБЛАЧНОЕ ХРАНИЛИЩЕ КАРТИНОК”
РАЗРАБОТАННОЕ НА ОСНОВЕ ЯЗЫКА PYTHON
И ФРЕЙМВОРКОВ DRF И PyQt5**

**ЮЖНО-САХАЛИНСК
2026**

СОДЕРЖАНИЕ

1.ВВЕДЕНИЕ	3
1.1 СЕРВЕРНАЯ ЧАСТЬ НА БАЗЕ DJANGO REST FRAMEWORK:	3
1.2 КЛИЕНТСКАЯ ЧАСТЬ НА БАЗЕ PYQT5:	3
2.ТЕХНИЧЕСКОЕ ЗАДАНИЕ	4
2.1. ВВЕДЕНИЕ	4
2.2 НАЗНАЧЕНИЕ РАЗРАБОТКИ	4
2.3. ТРЕБОВАНИЯ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ	5
2.3.1. ТРЕБОВАНИЯ К ФУНКЦИОНАЛЬНЫМ ХАРАКТЕРИСТИКАМ	5
2.3.1.1. ТРЕБОВАНИЯ К ИНСТРУМЕНТАЛЬНОЙ СРЕДЕ	5
2.3.1.2. ТРЕБОВАНИЯ К ПОЛЬЗОВАТЕЛЬСКОМУ ИНТЕРФЕЙСУ	5
2.3.1.3. ТРЕБОВАНИЯ К РЕАЛИЗУЕМЫМ ФУНКЦИЯМ	5
2.3.2. ТРЕБОВАНИЯ К НАДЕЖНОСТИ	6
2.3.3. ТРЕБОВАНИЯ К СОСТАВУ ТЕХНИЧЕСКИХ СРЕДСТВ	6
2.3.4. ТРЕБОВАНИЯ К СОВМЕСТИМОСТИ	6
3. ОБЩАЯ ЛОГИЧЕСКАЯ СХЕМА ПРОГРАММЫ	7
3.1 СХЕМА КЛИЕНТСКОГО ПРИЛОЖЕНИ	7
3.2 СХЕМА РАБОТЫ СЕРВЕРА	7
4.ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ	8
5. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ	8
6. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ	8

7. КАК РАБОТАЕТ ПРОГРАММА	9
7.1. ЗАПУСК ПРОГРАММЫ	9
7.2. ВЫБОР ИЗОБРАЖЕНИЯ	9
7.3. ОТПРАВКА ИЗОБРАЖЕНИЯ	9
7.4. ПОЛУЧЕНИЕ ИЗОБРАЖЕНИЙ	9
7.5. ПРОСМОТР ИЗОБРАЖЕНИЙ	10
8. ЛОГИЧЕСКИЕ ВЗАИМОСВЯЗИ	11
8.1. АРХИТЕКТУРА ПРОГРАММЫ	11
8.2. ВЗАИМОДЕЙСТВИЕ КОМПОНЕНТОВ	12
8.3. РАБОТА ИНТЕРФЕЙСА	12
9. ЛИСТИНГ ПРОГРАММЫ	12
9.1 ФАЙЛЫ DJANGO	12
9.1.1 ADMIN.PY	12
9.1.2 APPS.PY	12
9.1.3 MODELS.PY	12
9.1.4 SERIALIZERS.PY	13
9.1.5 TESTS.PY	13
9.1.6 URLS.PY	13
9.1.7 VIEWS.PY	13
9.1.8 ASGI.PY	14
9.1.9 PROJECT/URLS.PY	14
9.1.10 WSGI.PY	14
9.1.11 SETTINGS.PY	14
9.2 КОД КЛИЕНТСКОГО ПРИЛОЖЕНИЯ	17

1.ВВЕДЕНИЕ

Документация описывает разработанное программное обеспечение для передачи и получения изображений по сети с использованием клиент-серверной архитектуры. Проект реализован на языке Python с применением фреймворков PyQt5 и Django REST Framework.

Программное обеспечение состоит из двух основных частей:

1.1 СЕРВЕРНАЯ ЧАСТЬ НА БАЗЕ DRF:

- обработка HTTP-запросов;
- хранение изображений;
- предоставление REST API;
- сериализация данных.

1.2 КЛИЕНТСКАЯ ЧАСТЬ НА БАЗЕ PyQt5:

- графический интерфейс пользователя;
- выбор изображения;
- отправка изображений на сервер;
- получение списка изображений;
- отображение изображений.

Программа реализует взаимодействие между клиентом и сервером через HTTP-запросы библиотеки requests.

2.ТЕХНИЧЕСКОЕ ЗАДАНИЕ

2.1. ВВЕДЕНИЕ

Наименование разработки: «Облачное хранилище изображений».

Программа предназначена для демонстрации клиент-серверного взаимодействия между приложением PyQt5 и сервером Django REST Framework.

2.2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

Программа предназначена для:

- загрузки изображений на сервер;
- хранения изображений;
- получения списка загруженных изображений;
- просмотра изображений через графический интерфейс.

2.3. ТРЕБОВАНИЯ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ

2.3.1. Требования к функциональным характеристикам

2.3.1.1. Требования к инструментальной среде

1. Python версии не ниже 3.10.
2. Используемые библиотеки:
 - a. PyQt5;
 - b. Django;
 - c. Django REST Framework;
 - d. Pillow;
 - e. requests.

2.3.1.2. Требования к пользовательскому интерфейсу

1. Интерфейс должен содержать:
 - a. кнопки отправки и получения изображений;
 - b. поле ввода пароля;
 - c. список изображений;
 - d. область предпросмотра изображения.
2. Пользователь должен иметь возможность:
 - a. выбрать изображение;
 - b. отправить изображение;
 - c. получить изображения с сервера;
 - d. просмотреть выбранное изображение.

2.3.1.3. Требования к реализуемым функциям

1. При запуске клиентское приложение:
 - a. создает графический интерфейс;
 - b. подключает обработчики событий.
2. При выборе изображения:
 - a. открывается QFileDialog;
 - b. путь сохраняется в переменную image_path.
3. При отправке изображения:
 - a. выполняется POST-запрос к серверу;
 - b. изображение передается через multipart/form-data.
4. При получении изображений:
 - a. выполняется GET-запрос;
 - b. список изображений отображается в QListWidget.
5. При выборе изображения из списка:
 - a. изображение загружается;
 - b. выполняется отображение в QLabel.

2.3.2. Требования к надежности

1. Программа должна обрабатывать ошибки:
 - а. отсутствия изображения;
 - б. отсутствия соединения с сервером;
 - с. пустого списка изображений.
2. Интерфейс не должен завершаться аварийно при ошибках запросов.

2.3.3. Требования к составу технических средств

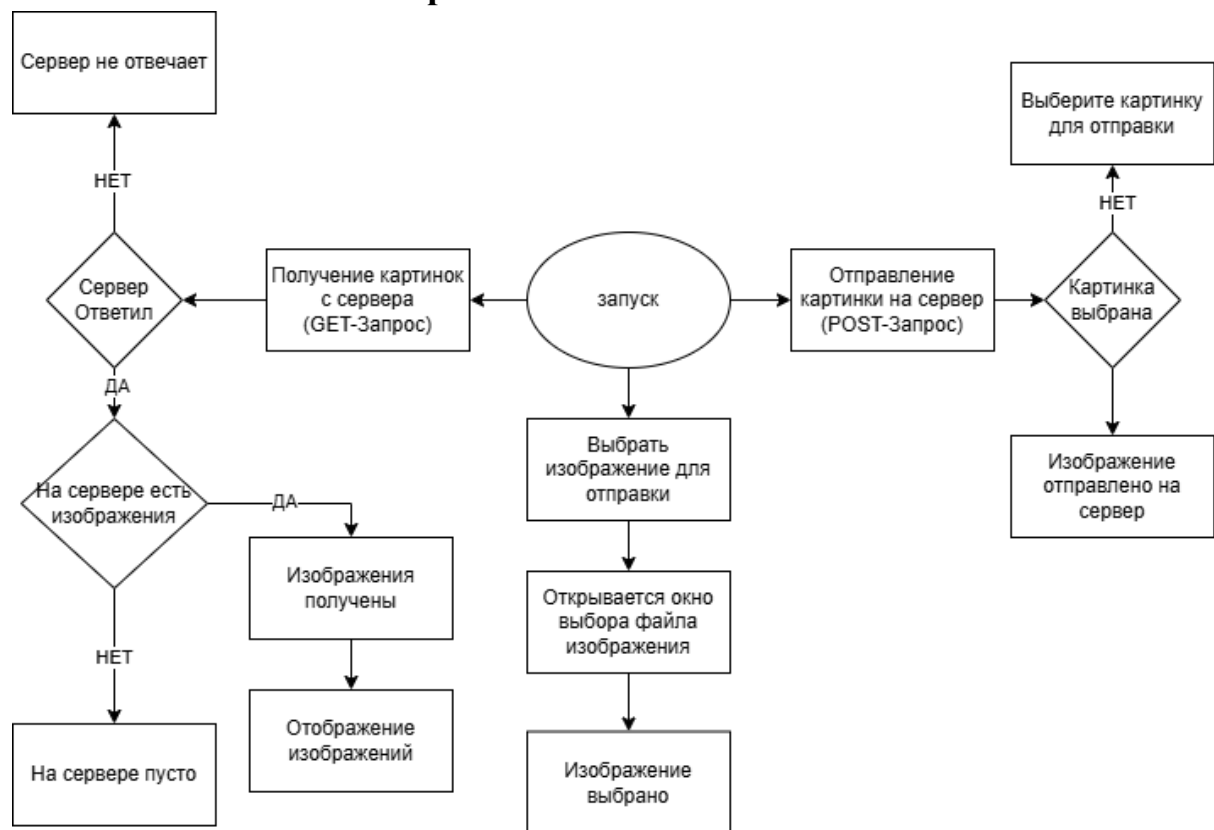
1. ОС Windows/Linux.
2. Наличие Python и необходимых библиотек.
3. Наличие сетевого соединения между клиентом и сервером.

2.3.4. Требования к совместимости

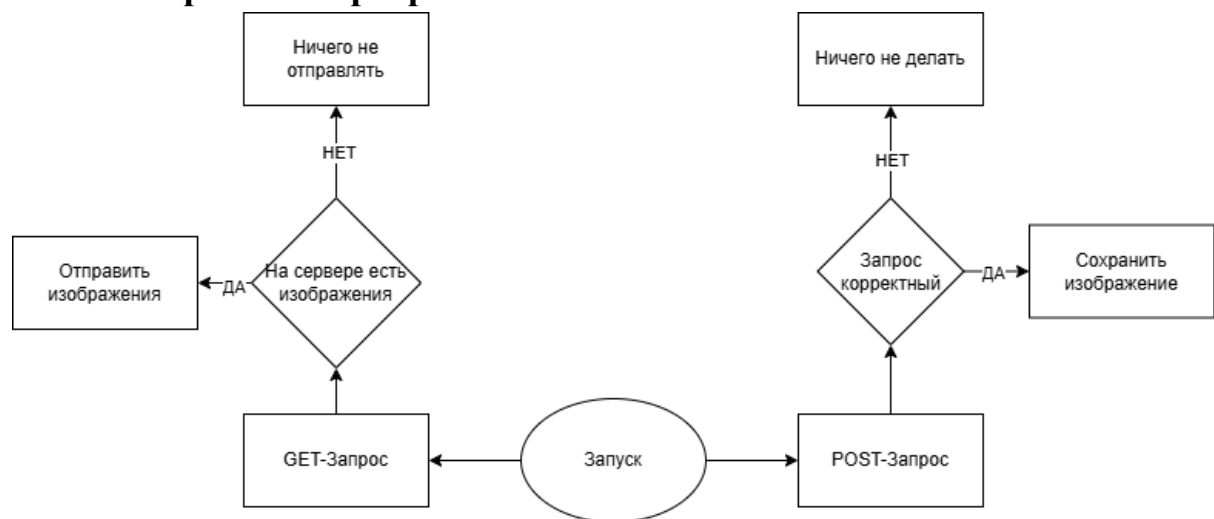
1. Сервер должен поддерживать HTTP-запросы.
2. Передача данных выполняется через REST API.

3. ОБЩАЯ ЛОГИЧЕСКАЯ СХЕМА ПРОГРАММЫ

3.1 Схема клиентского приложения



3.2 Схема работы сервера



4. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ

№	Функция/Класс	Описание
1	MyWindow(QWidget)	Главное окно программы

2	<code>initUI()</code>	Создание интерфейса
3	<code>change_text1()</code>	Отправка изображения
4	<code>change_text2()</code>	Получение изображений
5	<code>item_clicked()</code>	Выбор изображения из полученных
6	<code>pick_file()</code>	Выбор файла изображения
7	<code>Image(models.Model)</code>	Модель изображения
8	<code>ImageSerializer</code>	Сериализация данных
9	<code>requests.post()</code>	Отправка POST-Запроса
10	<code>requests.get()</code>	Отправка GET-Запроса

5. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ

№	Переменная	Описание
1	<code>self.password_field.text()</code>	Введенный пароль
2	<code>self.image_path</code>	Путь к изображению
3	<code>item.text()</code>	URL изображения
4	<code>r.content</code>	Содержимое ответа сервера
5	<code>files={"image": f}</code>	Отправляемое изображение
6	<code>self.button1.clicked.connect(self.change_text1)</code>	Инициализация обработчика нажатия кнопки
7	<code>self.button2.clicked.connect(self.change_text2)</code>	Инициализация обработчика нажатия кнопки
8	<code>self.image_button.clicked.connect(self.pick_file)</code>	Инициализация обработчика нажатия кнопки
9	<code>self.images_group.itemClicked.connect(self.item_clicked)</code>	Пользователь выбирает пункт списка
10	<code>self.button1 = QPushButton(f"отправить")</code>	Кнопка отправить
11	<code>self.button2 = QPushButton(f"получить")</code>	Кнопка получить

12	<code>self.images_group = QListWidget()</code>	Список получаемых от сервера изображений
13	<code>self.image_button = QPushButton()</code>	Кнопка выбора изображения для отправки
14	<code>self.security = QGroupBox("введите пароль - (1)")</code>	Подсказка к паролю
15	<code>self.password = '1'</code>	Ожидаемый пароль
16	<code>self.security_layout = QVBoxLayout()</code>	Контейнер для поля ввода пароля
17	<code>self.password_field = QLineEdit()</code>	Поле ввода пароля
18	<code>self.widget3 = QLabel()</code>	Текст отображающий статус приложения
19	<code>r = requests.get("http://127.0.0.1:8000/main/images/")</code>	Получаемые с сервера изображения
20	<code>x = requests.get(i['image'])</code>	Получение отдельного изображения с сервера
21	<code>scaled_pixmap = self.pixmap.scaled(200, 200, QtCore.Qt.KeepAspectRatio)</code>	Изображение отображаемое в клиенте
22	<code>app = QApplication(sys.argv)</code>	Инициализация приложения
23	<code>window = MyWindow()</code>	Основное окно
24	<code>self.pixmap = QPixmap("")</code>	Контейнер для отображаемого изображения
25	<code>image = models.ImageField(upload_to='images/')</code>	Поле изображения модели базы данных
26	<code>fields = ['image']</code>	Поля модели базы данных для сериализации
27	<code>urlpatterns = [path('main/', include(router.urls)),]</code>	Установка URL путей доступных на сервере
28	<code>queryset = Image.objects.all()</code>	Инициализация URL интерфейса сервера

29	<code>serializer_class = ImageSerializer</code>	Установка сериализатора данных для URL интерфейса сервера
30	<code>ALLOWED_HOSTS = []</code>	Снятие ограничений доступа на обращение к серверу
31	<code>admin.site.register(Image)</code>	Регистрация модели данных БД в админской панели сервера
32	<code>AUTH_PASSWORD_VALIDATORS</code>	Встроенные валидаторы пароля на сервере
33	<code>DATABASES</code>	Определение базы данных на сервере
34	<code>LANGUAGE_CODE</code>	Язык сервера
35	<code>TIME_ZONE</code>	Временной пояс сервера
36	<code>USE_I18N</code>	Определение интернационализации
37	<code>USE_TZ</code>	Включение работы с часовыми поясами на сервере
38	<code>STATIC_URL</code>	Статичный URL сервера
39	<code>DEFAULT_AUTO_FIELD</code>	Установка стандартного поля для модели базы данных
40	<code>WSGI_APPLICATION</code>	Указание точки входа на сервер для WSGI хостов
41	<code>TEMPLATES</code>	Установка общих шаблонов сервера

42	ROOT_URLCONF	Указание пути к главному URL файлу сервера
43	MIDDLEWARE	Определение обработчиков запросов проходящих на и от сервера
44	INSTALLED_APPS	Установленные приложения сервера
45	CSRF_COOKIE_HTTPONLY	Разрешение хостам считывать CSRF токени из cookie
46	CSRF_COOKIE_SECURE	Разрешение отправки куки хостам использующим протокол HTTP
47	APPEND_SLASH	Включение переадресации пользователя обращающегося к странице без закрывающего слеша на страницу с закрывающим слешом
48	DEBUG	Включение режима отладки сервера
49	SECRET_KEY	Определение секретного ключа шифрования сервера
50	BASE_DIR	Определение корневой папки проекта

6. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ

№	Воздействие	Описание
1	self.widget3.setText()	Вывод статуса запроса
2	self.image.setPixmap()	Отображение изображений

3	<code>QListWidgetItem()</code>	Добавление изображения в список
4	<code>requests.post()</code>	Отправка изображения
5	<code>requests.get()</code>	Получение изображений
6	<code>window.show()</code>	Показ окна приложения пользователю

7. КАК РАБОТАЕТ ПРОГРАММА

7.1. Запуск программы

При запуске выполняются:

1. Запуск сервера Django: `os.system('python project/run.py')`
2. Создание `QApplication`.
3. Создание окна `MyWindow`.
4. Отображение интерфейса.

7.2. Выбор изображения

Пользователь нажимает кнопку:

```
self.image_button.clicked.connect(self.pick_file)
```

После чего:

- открывается `QFileDialog`;
- выбирается изображение;
- путь сохраняется в `self.image_path`.

7.3. Отправка изображения

Метод:

```
change_text1()
```

выполняет:

1. Проверку пароля.
2. Открытие файла изображения.
3. POST-запрос:
`requests.post("http://127.0.0.1:8000/main/images/", files={"image": f})`
4. Сервер сохраняет изображение.

7.4. Получение изображений

Метод:

```
change_text2()
```

выполняет:

1. GET-запрос:
`requests.get("http://127.0.0.1:8000/main/images/")`
2. Получение JSON-списка изображений.
3. Загрузка изображений.
4. Добавление элементов в `QListWidget`.

7.5. Просмотр изображения

При выборе изображения:

```
self.images_group.itemClicked.connect(self.item_clicked)
```

выполняется:

- загрузка изображения по URL;
- отображение в `QLabel`.

8. ЛОГИЧЕСКИЕ ВЗАИМОСВЯЗИ

8.1. Архитектура программы

Программа построена по клиент-серверной архитектуре:

Клиент:

- PyQt5 GUI;
- `requests`;
- обработка изображений.

Сервер:

- Django;
- Django REST Framework;
- `ImageField`;
- сериализация JSON.

8.2. Взаимодействие компонентов

Отправка изображения

PyQt5 → `requests.post()` → Django REST API → Image модель

Получение изображения

PyQt5 → `requests.get()` → Django REST API → JSON → `QListWidget`

8.3. Работа интерфейса

GUI работает через механизм сигналов и слотов PyQt5:

```
clicked.connect(...)
itemClicked.connect(...)
```

9. ЛИСТИНГ ПРОГРАММЫ

9.1 Файлы Django

9.1.1 Models.py

```
class Image(models.Model):
    image = models.ImageField(upload_to='images/')
```

9.1.2 Serializers.py

```
class ImageSerializer(serializers.ModelSerializer):
    class Meta:
        model = Image
        fields = ['image']
```

9.1.3 admin.py

```
from django.contrib import admin

from .models import Image

# Register your models here.

admin.site.register(Image)
```

9.1.4 apps.py

```
from django.apps import AppConfig

class ImageSenderConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'image_sender'
```

9.1.5 tests.py

```
from django.test import TestCase

# Create your tests here.
```

9.1.6 urls.py

```
from django.contrib import admin
from django.urls import path, include
from .views import (
    ImageViewSet
)
from rest_framework.routers import DefaultRouter
from django.conf import settings
from django.conf.urls.static import static
router = DefaultRouter()
router.register(r'images', ImageViewSet)
urlpatterns = [
    path('main/', include(router.urls))]
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

9.1.7 views.py

```
from django.shortcuts import render
from rest_framework import viewsets
from .models import Image
from .serializers import ImageSerializer
# Create your views here.
class ImageViewSet(viewsets.ModelViewSet):
    queryset = Image.objects.all()
    serializer_class = ImageSerializer
```

9.1.8 asgi.py

```
import os
from django.core.asgi import get_asgi_application
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'project.settings')
```

```
application = get_asgi_application()
```

9.1.9 wsgi.py

```
import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'project.settings')

application = get_wsgi_application()
```

9.1.10 project/urls.py

```
from django.contrib import admin

from django.urls import path, include

urlpatterns = [

    path('admin/', admin.site.urls),

    path("", include('image_sender.urls'))

]
```

9.1.11 settings.py

```
""" Django settings for project project.
```

Generated by 'django-admin startproject' using Django 5.2.11.

For more information on this file, see <https://docs.djangoproject.com/en/5.2/topics/settings/>

For the full list of settings and their values, see
<https://docs.djangoproject.com/en/5.2/ref/settings/> """

```
from pathlib import Path
```

Build paths inside the project like this: BASE_DIR / 'subdir'.

```
BASE_DIR = Path(file).resolve().parent.parent
```

Quick-start development settings - unsuitable for production

See <https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/>

SECURITY WARNING: keep the secret key used in production secret!

```
SECRET_KEY = 'django-insecure-!b%tpw-*)#8gsb0q+c_dok9m-!cymkto-45uvp5+x+)ia9yvuk'
```

SECURITY WARNING: don't run with debug turned on in production!

DEBUG = True

ALLOWED_HOSTS = []

Application definition

APPEND_SLASH = False CSRF_COOKIE_SECURE = False

CSRF_COOKIE_HTTPONLY = False INSTALLED_APPS = ['django.contrib.admin',
'django.contrib.auth', 'django.contrib.contenttypes', 'django.contrib.sessions',
'django.contrib.messages', 'django.contrib.staticfiles', 'rest_framework', 'image_sender',]

MIDDLEWARE = ['django.middleware.security.SecurityMiddleware',
'django.contrib.sessions.middleware.SessionMiddleware',
'django.middleware.common.CommonMiddleware', #
'django.middleware.csrf.CsrfViewMiddleware',
'django.contrib.auth.middleware.AuthenticationMiddleware',
'django.contrib.messages.middleware.MessageMiddleware',
'django.middleware.clickjacking.XFrameOptionsMiddleware',]

ROOT_URLCONF = 'project.urls'

TEMPLATES = [{ 'BACKEND': 'django.template.backends.django.DjangoTemplates',
'DIRS': [], 'APP_DIRS': True, 'OPTIONS': { 'context_processors': [
'django.template.context_processors.request', 'django.contrib.auth.context_processors.auth',
'django.contrib.messages.context_processors.messages',], }, },]

WSGI_APPLICATION = 'project.wsgi.application'

Database

<https://docs.djangoproject.com/en/5.2/ref/settings/#databases>

DATABASES = { 'default': { 'ENGINE': 'django.db.backends.sqlite3', 'NAME': BASE_DIR /
'db.sqlite3', } }

Password validation

<https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-validators>

AUTH_PASSWORD_VALIDATORS = [{ 'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator', }, { 'NAME':
'django.contrib.auth.password_validation.MinimumLengthValidator', }, { 'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator', }, { 'NAME':
'django.contrib.auth.password_validation.NumericPasswordValidator', },]

Internationalization

<https://docs.djangoproject.com/en/5.2/topics/i18n/>

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True

Static files (CSS, JavaScript, Images)

<https://docs.djangoproject.com/en/5.2/howto/static-files/>

STATIC_URL = 'static/'

Default primary key field type

<https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field>

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

9.2 Код Клиентского приложения

```
import os
import sys
import requests from PIL
import Image from PyQt5.QtWidgets
import ( QApplication, QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTextEdit,
QLabel, QGroupBox, QLineEdit, QFileDialog, QListWidget, QListWidgetItem)

from PyQt5.QtGui import QImage, QPixmap import json from PIL import Image from
PyQt5 import QtCore

class MyWindow(QWidget): def init(self): super().init() self.initUI()

def initUI(self):
    over_main_layout = QVBoxLayout()
    main_layout = QHBoxLayout()
    over_main_layout.addLayout(main_layout)

    self.button1 = QPushButton(f"отправить")
    self.button2 = QPushButton(f"получить")
    self.button1.clicked.connect(self.change_text1)
    self.button2.clicked.connect(self.change_text2)
    self.images_group = QListWidget()
    self.images_group.itemClicked.connect(self.item_clicked)
    self.image = QLabel()
    self.image_button = QPushButton()
    self.image_button.setText('выбрать изображение для отправки')
    self.image_button.clicked.connect(self.pick_file)
```

```

self.widget3 = QLabel()
self.security = QGroupBox("введите пароль - (1)")
self.security_layout = QVBoxLayout()
self.security.setLayout(self.security_layout)
self.password_field = QLineEdit()
self.security_layout.addWidget(self.password_field)
self.security_layout.addWidget(self.widget3)
self.security_layout.addWidget(self.image_button)
self.security_layout.addWidget(self.images_group)
self.security_layout.addWidget(self.image)
self.password = '1'
# self.widget1 = self.create_widget("отправить на сервер", 1)
# self.widget2 = self.create_widget("получить с сервера", 2)

self.pixmap = QPixmap('')

main_layout.addWidget(self.button1)
main_layout.addWidget(self.button2)

over_main_layout.addWidget(self.security)
self.setLayout(over_main_layout)
self.setWindowTitle('Два виджета с кнопками и текстом')
self.setGeometry(300, 300, 600, 600)

def create_widget(self, title, widget_num):
    group_box = QGroupBox(title)
    layout = QVBoxLayout()
    if widget_num == 1:
        self.text_edit1 = QTextEdit()
        self.text_edit1.setPlaceholderText(f"Текст для {title}")
        layout.addWidget(self.text_edit1)
    else:
        self.text_edit2 = QTextEdit()
        self.text_edit2.setPlaceholderText(f"Текст для {title}")
        layout.addWidget(self.text_edit2)

    button = QPushButton(f"Изменить текст в {title}")

    if widget_num == 1:
        button.clicked.connect(self.change_text1)
    else:
        button.clicked.connect(self.change_text2)

```

```

        layout.addWidget(button)
        group_box.setLayout(layout)

    return group_box
# , 'password' : self.password_field.text()}
def change_text1(self):
    try:
        if self.password_field.text() != self.password:
            self.widget3.setText('введите пароль')
            return None
        with open (self.image_path[0], 'rb') as f:
            print(f)
            print('-----')
            r = requests.post("http://127.0.0.1:8000/main/images/",
files = {"image": f})
            self.widget3.setText('изображение отправлено')
            print(r.text)
    except:
        self.widget3.setText('выберите изображение для отправки')
        # self.widget3.setText(r.text)

def change_text2(self):
    r = requests.get("http://127.0.0.1:8000/main/images/")
    try:
        if not json.loads(r.content):
            self.widget3.setText('на сервере пусто')
            r1 = requests.get(json.loads(r.content)[0]['image'])
    except:
        self.widget3.setText('ошибка получения(вероятно на сервере
пусто)')
    for i in json.loads(r.content):
        x = requests.get(i['image'])
        self.pixmap.loadFromData(x.content)
        scaled_pixmap = self.pixmap.scaled(200, 200,
QtCore.Qt.KeepAspectRatio)
        self.image.setPixmap(scaled_pixmap)
        QListWidgetItem(i['image'], self.images_group)
        # self.text_edit2.setText(r.text)
def item_clicked(self, item):
    print(item.text())
    x = requests.get(item.text())

```

```

        self.pixmap.loadFromData(x.content)
        scaled_pixmap = self.pixmap.scaled(200, 200,
QtCore.Qt.KeepAspectRatio)
        self.image.setPixmap(scaled_pixmap)
def pick_file(self):
    self.image_path = QFileDialog().getOpenFileName(
        self,
        'выберите изображение',
        '',
        'image (*.png *.jpg)'
    )
    self.widget3.setText('изображение выбрано')
    # self.image = Image.open(fp=self.image_path[0])
    # print(self.image)
    # self.image.setNameFilter("image (*.png *.jpg)")

```

```

if name == 'main': os.system('python project/run.py') app = QApplication(sys.argv) window
= MyWindow() window.show() sys.exit(app.exec_())

```